01. Foundations of Parallel Computing & Performance Measures

This chapter introduces the fundamental concepts, performance metrics, and basic patterns (skeletons) used in parallel computing to evaluate and structure concurrent execution.

1. Performance Metrics

To evaluate the quality and efficiency of a parallel system, we rely on several quantitative metrics:

1.1 Latency (L)

[cite_start]Latency is the time required to complete a single task from its beginning to its end[cite: 1]. [cite_start]In a system composed of multiple stages (e.g., a pipeline), it is the sum of the times spent in each individual stage[cite: 1].

1.2 Service Time (T_s)

[cite_start]Service Time is the time interval between the emission of two consecutive results[cite: 1]. It dictates the maximum throughput of the system. * In a sequential execution, T_s is equal to the Latency. * [cite_start]In a parallel system (like a pipeline), the overall service time is bottlenecked by the slowest stage: $T_s = \max\{T_{s_1}, T_{s_2}, \ldots, T_{s_k}\}$[cite: 1].

1.3 Completion Time (T_c)

[cite_start]The total time required to process m tasks[cite: 1]. [cite_start]Given a pipeline or a generic parallel system, the initial phase involves filling the system (paying the latency L), and subsequently, results are emitted every T_s[cite: 1].

$$T_c = L + (m-1)T_s$$

[cite_start]For a very large number of tasks ($m \gg 1$), the Completion Time can be approximated as $T_c \approx mT_s$[cite: 1].

1.4 Speedup (S_p)

[cite_start]Speedup measures the relative performance gain of executing a program on n parallel processing elements compared to a sequential execution[cite: 1].

$$S_p(n) = \frac{T_{seq}}{T_{par}(n)}$$

Where T_{seq} is the completion time of the best sequential algorithm and $T_{par}(n)$ is the completion time of the parallel algorithm using n workers. * [cite_start]Ideally, we seek **linear speedup** ($S_p(n) = n$)[cite: 1]. * [cite_start]Speedup is theoretically bounded by the critical path: $S_p \leq \frac{\text{Work}}{\text{Span}}$[cite: 1].

1.5 Efficiency (E)

[cite_start]Efficiency measures how well the parallel resources are being utilized[cite: 1]. [cite_start]It is defined as the ideal parallel time divided by the actual parallel time, or equivalently, Speedup divided by the number of workers n[cite: 1].

$$E(n) = \frac{T_{id}(n)}{T(n)} = \frac{T_{seq}}{n \cdot T_{par}(n)} = \frac{S_p(n)}{n}$$

[cite_start]An ideal efficiency is 1 (or 100%), corresponding to linear speedup[cite: 1].

---

2. Laws of Scalability

2.1 Amdahl's Law

Amdahl's law provides a theoretical upper bound on the speedup of a program when only a fraction of it can be parallelized. [cite_start]If f is the strictly sequential fraction of the program (e.g., $10\% = 0.10$), the maximum speedup is bounded by $\frac{1}{f}$ (e.g., $10\times$) regardless of how many cores n are added[cite: 1].

$$S_p(n) \leq \frac{1}{f + \frac{1-f}{n}} \xrightarrow{n\to\infty} \frac{1}{f}$$

2.2 Gustafson's Law

[cite_start]Gustafson's Law counteracts the pessimism of Amdahl's Law by observing that as computing power increases, the size of the problem typically increases as well[cite: 1]. [cite_start]The work required for the parallel part grows much faster than the sequential part[cite: 1]. [cite_start]If the sequential fraction f decreases as the data size grows, the system can achieve near-linear speedup for massive datasets[cite: 1].

---

3. Basic Parallel Patterns

Parallel patterns (or algorithmic skeletons) provide high-level abstractions to structure concurrent computations, avoiding low-level lock and thread management.

3.1 Pipeline

[cite_start]A computation is divided into k sequential stages connected by communication channels[cite: 1]. Each stage processes a data item and passes it to the next. * **Service Time (T_s):** Bounded by the slowest stage. [cite_start]$T_s = \max_{i=1\ldots k}\{T_{s_i}\}$[cite: 1]. * **Latency ($L$):** Sum of the times of all stages. [cite_start]$L = \sum_{i=1}^{k} T_{s_i}$[cite: 1]. * [cite_start]**Optimization:** If a stage is a bottleneck, its service time can be lowered by parallelizing that specific stage internally (e.g., turning it into a Farm)[cite: 1].

3.2 Farm (Task Farm / Master-Worker)

[cite_start]A Farm replicates the execution of a single function over multiple workers[cite: 1]. It is ideal for embarrassingly parallel workloads where tasks are completely independent. * [cite_start]**Structure:** Consists of an *Emitter* (Scheduler) that distributes tasks, a set of *Workers* that compute the tasks independently, and a *Collector* that gathers the results[cite: 1]. * **Service Time (T_s):** If the sequential service time is T_w and we have n_w workers, the ideal parallel service time is $T_s = \frac{T_w}{n_w}$[cite: 1]. * [cite_start]**Efficiency:** A farm is highly efficient, but scheduling overhead and the sequential nature of Emitter/Collector can become bottlenecks if the communication time dominates the computation time[cite: 1].

---

4. Hardware & Context

When designing parallel applications, the underlying hardware heavily influences the chosen mechanism and abstraction: * **Memory Architectures:** Threads can communicate via Shared Memory (requiring synchronization primitives like locks, condition variables, or barriers) or via Message Passing (Point-to-Point or Collective communications)[cite: 1]. * [cite_start]**Grain Size (Granularity):** The ratio of computation time to communication time[cite: 1]. [cite_start]A coarse-grained computation (high computation-to-communication ratio) is generally necessary to offset the overhead of data splitting and dispatching[cite: 1]. * **Excess Parallelism (Oversubscription):** Spawning more parallel activities than the available hardware contexts[cite: 1]. It allows the OS/runtime to hide latency (e.g., if one thread blocks for I/O, another can be scheduled), avoiding idle cores[cite: 1].

02. Structured Parallel Programming & Algorithmic Skeletons

This chapter delves into Structured Parallel Programming (SPP) and Algorithmic Skeletons. The core philosophy of this approach is the **separation of concerns**: isolating the purely sequential computational logic (business logic) from the parallel coordination and communication mechanisms.

1. The Algorithmic Skeleton Framework

Introduced by Murray Cole, **Algorithmic Skeletons** are recurring parallel paradigms abstracted into reusable, parameterizable templates. They provide high-level parallel constructs that hide the underlying complexity of thread management, synchronization, and message passing.

Skeletons are typically divided into two macro-categories based on the nature of the parallelism they exploit:

1. **Stream Parallel Skeletons:** Operate on streams of data items (tasks) flowing through the system. Examples: `pipeline`, `farm`.
2. **Data Parallel Skeletons:** Operate on large, partitioned data structures where the same computation is applied to different chunks simultaneously. Examples: `map`, `reduce`, `stencil`.

---

2. Stream Parallel Skeletons

Stream parallel skeletons model systems that process a continuous flow of input tasks. The execution is driven by the availability of new data tokens.

2.1 The Pipeline Skeleton (`pipe`)

The `pipe` skeleton connects k stages $S_1, S_2, \ldots, S_k$ in a sequence. The output stream of stage S_i becomes the input stream of stage S_{i+1}.

- **Semantics:** $\text{pipe}(S_1, S_2)(x) = S_2(S_1(x))$ extended over a stream.
- **Performance Model:**
  - **Service Time (T_s):** Bottlenecked by the slowest stage. $T_s = \max_{i=1\ldots k}\{T_{S_i}\}$
  - **Latency (L):** The sum of the times taken by all individual stages. $L = \sum_{i=1}^{k} T_{S_i}$
- **Optimization:** If a stage is identified as a bottleneck, it must be parallelized (e.g., by wrapping it in a `farm` or a nested `pipe`) to lower its service time and improve the overall throughput.

2.2 The Farm Skeleton (`farm`)

The `farm` skeleton models independent task execution (embarrassingly parallel). It consists of three entities:

1. **Emitter (E):** Fetches data from the input stream and dispatches it to the workers.
2. **Workers ($W_1 \ldots W_n$):** A pool of threads/processes computing the identical function f on different input data.
3. **Collector (C):** Gathers results from the workers and forwards them to the output stream.

- **Performance Model:**
  - Let T_E be the time the Emitter takes to schedule a task, T_W the time a worker takes to compute f, and T_C the time the Collector takes to gather a result.
  - **Service Time (T_s):** $T_s = \max\left\{T_E, \frac{T_W}{n}, T_C\right\}$
  - **Latency (L):** $L = T_E + T_W + T_C$

- **Scalability Bottleneck:** The Emitter and Collector are strictly sequential components. As the number of workers n increases, $\frac{T_W}{n}$ decreases. Eventually, T_E or T_C will become the maximum value, causing the speedup to plateau. The maximum useful number of workers is $n_{max} = \frac{T_W}{\max(T_E, T_C)}$.

---

3. Data Parallel Skeletons

Data parallel skeletons apply operations across partitions of a single, large data structure (like vectors, matrices, or graphs).

3.1 The Map Skeleton (`map`)

The `map` skeleton takes an array (or list) and applies a purely sequential function f to each element independently.

- **Semantics:** $\text{map}(f)([x_1, x_2, \ldots, x_n]) = [f(x_1), f(x_2), \ldots, f(x_n)]$
- **Execution:** The runtime partitions the array into chunks, assigns chunks to different processing elements (PEs), computes the chunks in parallel, and merges the results.
- **Completion Time (T_c):** For an array of size N distributed over n workers, the ideal completion time is $T_c = \frac{N \cdot T_f}{n} + T_{scatter} + T_{gather}$.

3.2 The Reduce Skeleton (`reduce`)

The `reduce` skeleton takes an array and aggregates its elements into a single value using a binary operator $\oplus$. For the skeleton to yield deterministic results regardless of the parallel execution tree, the operator $\oplus$ **must be associative**.

- **Semantics:** $\text{reduce}(\oplus)([x_1, x_2, \ldots, x_n]) = x_1 \oplus x_2 \oplus \cdots \oplus x_n$
- **Execution:** Typically implemented using a parallel tree reduction.
- **Completion Time (T_c):** With n workers, the reduction tree has a depth of $\log_2(n)$. Thus, parallel time scales logarithmically rather than linearly in the number of processors.

---

4. Operational Semantics for Skeletons

To formalize the behavior of these skeletons without ambiguity, we use operational semantics modeled as transition systems. States represent the current configuration of the computation, and transitions ($\rightarrow$) describe computation steps.

Let $\langle x, \tau \rangle$ represent an input stream where x is the current token and τ is the rest of the stream. Let Δ represent the inner computation of a skeleton (e.g., the worker function in a farm).

1. **Sequential execution (`seq`):**

   Applies a function f to a stream.

$$\langle \text{seq } f, \langle x, \tau \rangle \rangle \rightarrow \langle f(x) \rangle :: \langle \text{seq } f, \langle \tau \rangle \rangle$$

2. **Farm execution (`farm`):**

   The farm processes the first element of the stream and concurrently processes the rest.

$$\langle \text{farm } \Delta, \langle x, \tau \rangle \rangle \rightarrow \langle \Delta(x) \rangle \parallel \langle \text{farm } \Delta, \langle \tau \rangle \rangle$$

(Note: The exact formalization can include explicit rules for Emitter scheduling and Collector gathering depending on the specific transition system used).

---

5. Implementation in Real Frameworks (e.g., C++ FastFlow)

In modern C++ environments, implementing these mathematical models effectively requires low-overhead runtime supports. Frameworks like FastFlow implement stream parallel skeletons using **lock-free** and **wait-free** queues (e.g., Single-Producer Single-Consumer queues for pipelines).

- In a FastFlow `ff_pipeline`, each stage is a C++ object executing a sequential loop, pulling data from a lock-free input queue and pushing to an output queue.
- In a `ff_farm`, the Emitter acts as a load balancer, distributing pointers to memory allocations rather than copying the data itself, thereby minimizing T_E and extending the scalability limits defined by Amdahl's Law.